

Tree Search

Task Author: Teimuraz Toloraia

Subtask 1. Constraint in this subtask means that we can ask a question for every vertex. Let's denote the special vertex as X . Since we will receive 1 as a result only from X and its ancestors, X will be the deepest vertex that returns 1.

Subtask 2. In this subtask we are given a "line" tree. Which means that if we ask a question for a vertex k , we will receive 1 if $k \leq X$ and 0 otherwise. This gives us a simple condition for binary search. This will need $\log_2(n)$ questions.

Subtask 3. Condition of this subtask gives us a complete binary tree (meaning that all levels are filled completely except maybe the last one). Height of a complete binary tree is $\log_2(n)$. Let's first ask questions about vertices 2 and 3. If both of them return 0, then the answer is 1. Otherwise one of them will return 0 and one of them will return 1. If for example a question for vertex 3 returns 0 and vertex 2 returns 1, then we know for sure that X will not be equal to 1, 3 or any other descendant of 3. It will be 2 or one of its descendants. Now we have the same problem we had at the start but the root of the remaining tree is 2. Thus using 2 questions we have reduced the height of the tree by one. And now we can repeat this step until the height of the tree becomes one. To achieve this we will need $2 * \log_2(n)$ questions.

Subtask 4. When we ask a question it divides the tree into 2 parts, the subtree of the asked vertex and everything else. Depending on the answer X might be in only one of these parts and we do not care about the other one going forward. So it is intuitive to make the bigger of the 2 parts as small as possible to reduce the size of the tree as quickly as we can. This strategy would be enough to solve the problem because it turns out that we can always choose the vertex in a way that makes the size of the bigger part be at most two thirds of the whole tree. We can prove it like this:

Let n denote the size of the tree. We want to find the vertex with subtree size s satisfying $\frac{1}{3}n \leq s \leq \frac{2}{3}n$. To begin, let's pick the root as the current vertex. We are going to maintain the condition that subtree size of the current vertex is bigger than $\frac{2}{3}n$. Subtree of the root is the whole tree so its size is bigger than $\frac{2}{3}n$. Now let's look at its children. If one of them satisfies the above condition we are done. Otherwise since both of them can't be smaller than $\frac{1}{3}n$, one of them has to be bigger than $\frac{2}{3}n$. So we can make the child with a bigger subtree the current vertex while maintaining the set condition. And now we can repeat this step until one of the children satisfies the condition we needed.

If we make divisions like this, the size of the relevant tree reduces by a factor of at least 1.5. Which gives us upper limit of $\log_{1.5}(n)$ questions until the size gets reduced to 1.

Teleporters

Task Author: Host Scientific Committee

Let us examine what happens after teleportation if Anna was at coordinate A and chose teleporter x and Beka was at coordinate B and chose teleporter y . Anna goes to coordinate $2c_x - A$, and Beka goes to $2c_y - B$. The distance between them becomes $abs((2c_x - A) - (2c_y - B))$, which means it changes by $abs(2(c_x - c_y))$.

Let us consider any valid answer and the sequence of teleportations in it.

$a_1, a_2, \dots, a_k$ — ordered sequence of teleporters used by Anna.

$b_1, b_2, \dots, b_k$ — ordered sequence of teleporters used by Beka.

Now create a new sequence ab by reversing sequence b and concatenate it with a ($a_1, a_2, \dots, a_k, b_k, b_{k-1}, \dots, b_1$).

We can rephrase the problem statement as follows:

Find a sequence ab of teleporters using which Anna could end up at Beka's coordinate and minimize the value of $\max_{1 \leq i \leq k} |f_{ab_i} - f_{ab_{2k-i+1}}|$.

Now we calculate Anna's coordinate after each step:

$$coord_1 = 2ab_1 - A$$

$$coord_2 = 2ab_2 - coord_1 = (2ab_2 - 2ab_1) + A$$

$$coord_3 = 2ab_3 - coord_2 = 2ab_3 - 2ab_2 + 2ab_1 - A$$

$$coord_4 = 2ab_4 - coord_3 = (2ab_4 - 2ab_3) + (2ab_2 - 2ab_1) + A$$

$$coord_{2k} = 2 \sum_{i=1}^k (ab_{2i} - ab_{2i-1}) + A = (2 \sum_{i=1}^k ab_{2i} - 2 \sum_{i=1}^k ab_{2i-1}) + B$$

From the last equation we can see that reordering teleporters of the same parity would not change the final destination.

Let's reverse the order of teleporters with even indices and rephrase the statement one more time:

Find a sequence ab' of teleporters using which Anna could end up at Beka's coordinate and minimize the

value of $\max_{1 \leq i \leq k} |f_{ab'_{2i}} - f_{ab'_{2i-1}}|$.

Now we can match the pairs of neighboring indices in ab and consider those teleportations together. After a pair of teleportations (ab_{2i-1}, ab_{2i}) Anna's increases by $2(ab_{2i} - ab_{2i-1})$, and the discomfort from the

initial statement would be equal to $|f_{ab_{2i}} - f_{ab_{2i-1}}|$.

Because the distance change is derived only from the distance between chosen teleporters, we do not need to maintain any coordinates. The discomfort during the minute also depends only on the teleporters' frequencies.

Sort the teleporters in non-decreasing order of frequencies. **We will only use this order.**

Claim 1: It is always possible to achieve the answer, even if Anna and Beka only use adjacent (in the sorted array) teleporters for simultaneous teleportations.

Proof 1: Take any sequence of teleportations which achieves the answer. For two teleporters x and y , $x \leq y$, their distance change (as described in the second rephrased statement) — $2(c_x - c_y)$ — is

achievable by using only adjacent teleporters in each minute — $\sum_{i=1}^{y-x} 2(c_{x+i} - c_{x+i-1})$. Substitute the pairs

which are not adjacent in the solution sequence and we get the desired one.

By this substitution the discomfort in each minute cannot become higher:

$$f_y - f_x = \sum_{i=1}^{y-x} (f_{x+i} - f_{x+i-1}).$$

$$origDiscomfort = \sum_{i=1}^{y-x} newDiscomfort_i.$$

Since the array is ordered in increasing order of frequencies, all values above are non-negative, which means $newDiscomfort_i \leq origDiscomfort$ for any i .

Since for each pair of teleporters we only care about the distances between them and the differences in their frequencies, we calculate these values between adjacent ones (other ones are not needed because of claim 1) and store them in arrays.

$$deltaFreq_i = f_{i+1} - f_i.$$

$$deltaCoord_i = 2(c_{i+1} - c_i).$$

Now it is also obvious that the order of simultaneous teleportations does not matter.

To make Anna and Beka meet, we need to make $X - Y$ equal to zero by adding or subtracting numbers from $deltaCoord$ to it. The answer then would be maximal $deltaFreq$ that was used.

Let S be a set of used indices of $deltaCoord$. It is necessary and enough to have $\gcd(S)$ be a divisor of $X - Y$ (using the Euclidean algorithm and maintaining the coefficients for each member we could construct the desired sum).

We construct a segment tree on the array of delta's.

In each node, for frequency F we store $\gcd$ of $deltaCoord_i$ -s, there $deltaFreq_i \leq F$. to avoid quadratic memory (and time) usage, we can store only different $\gcd$ -s. Since these $\gcd$ -s are non-increasing and each decrement makes them at least twice smaller, number of different $\gcd$ -s is $\log(10^9)$. To answer a query we just merge $O(\log(n))$ nodes' arrays of $\gcd$ -s and select the one which is a divisor of $X - Y$ and has smallest $deltaFreq$.

Time complexity: $O(n \log(n) \log(10^9))$.

Memory complexity: $O(n \log(10^9))$.

Opening Offices

Task Author: Host Scientific Committee

Subtask 1: $M, N \leq 2$.

There are only three different variants (and their rotations) of a city. There are three T types for each. Hence, we can count all the optimal configurations by hand.

Time complexity: $O(1)$.

Subtask 2: $N = 1$.

Since the city is on a straight line, any set of buildings satisfies the requirement (tour length will be the distance between the leftmost and rightmost of the chosen buildings multiplied by two. Both during day and night).

For $T = 1$, the number of ways to choose a non-empty set from M buildings is $2^M - 1$.

For $T = 2$, the number of ways to choose two buildings is $\binom{M}{2}$.

For $T = 3$, the number of ways to choose three buildings is $\binom{M}{3}$.

Time complexity: $O(M)$.

Subtask 3: $T = 2$; $N, M \leq 50$.

The length of a tour during daytime is the Manhattan distance (sum of the differences of the corresponding coordinates) between the two offices multiplied by two. For a pair of buildings, they satisfy the requirement iff the tree path between them does not contain any redundant edges. We will call pairs like this optimal. For this subtask, it is enough to go over all possible pairs and check if they are optimal (run a dfs/bfs from one of them and maintain the path length).

Time complexity: $O((NM)^2)$

Subtask 4: $T = 2$; $N, M \leq 1000$

Let us select an arbitrary pair (A, B) .

Claim 4.1: For the pair to be optimal, the path between A and B cannot include edges with opposite directions.

Proof 4.1: The tour is double the path from A to B . If we go from A to B , the Manhattan distance must decrease by one after each passed edge. That is only possible when the direction for both vertical and horizontal edges does not change.

Observation 4.1: If X is a building on a path between A and B , (A, X) and (B, X) pairs are optimal iff (A, B) is optimal.

We calculate a dp for all directions: ur, rd, dr, lu — up/right, right/down, down/left, left/up.

In $dp[i][j]$, we store the number of buildings in the corresponding sub-quadrant, which form an optimal pair together with (i, j) . transition looks like this:

$$\begin{aligned} dpQuad[i][j][lu] = & isEdge((i, j); (i - 1, j)) * dpQuad[i - 1][j][lu] \\ & + isEdge((i, j); (i, j - 1)) * dpQuad[i][j - 1][lu]. \end{aligned}$$

It's important to note that we use the observation above for the dp transition.

Time complexity: $O(NM)$.

Subtask 5: $T = 3$; $N, M \leq 20$.

Let us select an arbitrary triplet (A, B, C) , which satisfies the requirements.

Claim 5.1: (A, B) , (B, C) , and (C, A) must all be optimal pairs.

Proof 5.1: WLOG let the daytime tour plan be $(A \rightarrow B, B \rightarrow C, C \rightarrow A)$. Each of these three paths at daytime is at least as good as at nighttime, so each must be optimal for their sum at nighttime to be the same as at daytime.

For this subtask, we can go over all the possible configurations of the offices and check requirements using the solution of the third subtask.

Time complexity: $O((NM)^3)$.

Subtask 6: $T = 3$; $N, M \leq 1000$.

Let D be the building from which the sum of distances to A , B , and C is minimal. If there are several such buildings, one of the offices must be on the path between two others. In that case, let D be that office. Now, each triplet corresponds to precisely one building D , so we can count and sum up the number of triplets for each D to get the answer.

Claim 6.1: if (a, b) , (a, c) , (b, c) are optimal pairs, then it is required that the paths from a to b and c go in different directions both horizontally and vertically.

Proof 6.1: Assume the opposite. WLOG, let the shared direction be upward. If (a, b) contains an upward edge, then (b, a) contains a downward edge, so $(b, c) = (b, a) \cup (a, c)$ contains both downward and upward edges, which contradicts claim 4.1.

Each of the pairs (A, D) , (B, D) , (C, D) is optimal by claim 6.1.

We consider two cases:

1. One of the offices is built in the building D ;

Two other offices are an optimal pair, so they must be in different directions from the D office. More accurately, one must be up and right or left and up, and the second must have the opposite directions. We can use the dp from subtask 5 to calculate the number of such occurrences.

2. None of the offices are built in the building D ;

Claim 6.2: at most one of the paths $(D \rightarrow A, D \rightarrow B, D \rightarrow C)$ can have both vertical and horizontal edges.

Proof 6.2: Assume the opposite. WLOG $(D \rightarrow A)$ and $(D \rightarrow B)$ contain vertical and horizontal edges. They do not share the same direction because of claim 6.1, so all possible four directions occur in one of the two paths. That means $(D \rightarrow C)$ shares an edge direction with one of the other pairs, which contradicts claim 6.1.

We need one more dp — *dpLine*, similar to *dpQuad*, but only for buildings in the same row and column (count of buildings in four main directions). With these two, we can easily count the number of triplets for a fixed D . It is essential to keep track of duplicates.

Time complexity: $O(NM)$.

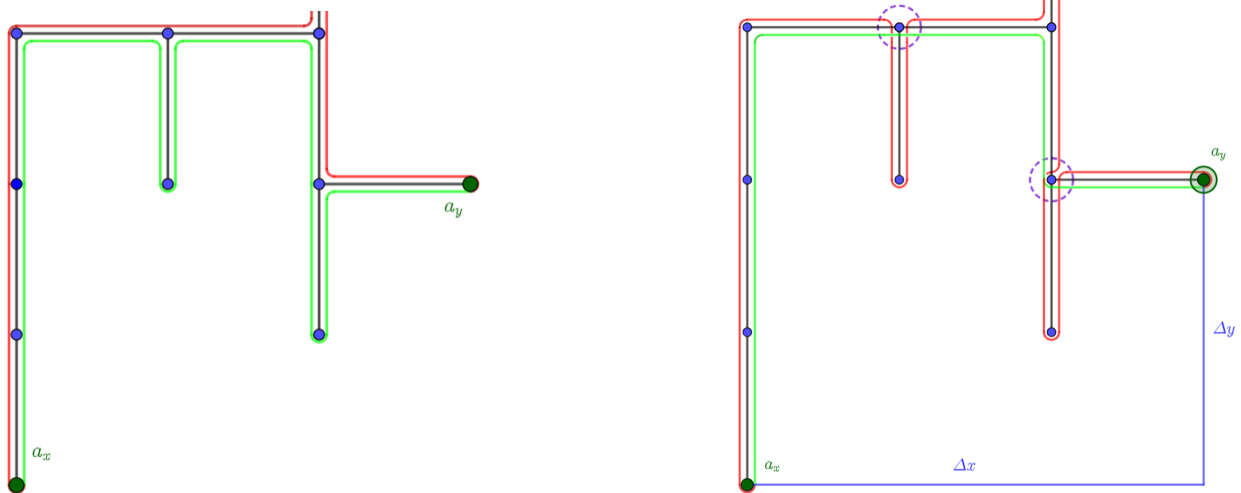
Subtask 7: $T = 1$; $N, M \leq 4$

Claim 7.1: The nighttime tour length is twice the size of all the offices' Steiner tree. Steiner tree of a set of vertices is a tree with the least amount of edges, which connects all the vertices in the set. In this problem, it is enough to take the union of the edges of all pairs in the set to find the Steiner tree.

Proof 7.1: Each edge in the Steiner tree must be crossed in both directions for a tour to be complete.

Claim 7.2: A set of offices $(a_1, a_2, \dots, a_k)$ meets the requirement (is optimal) iff all the pairs (a_i, a_j) are optimal.

Proof 7.2: Assume the opposite. Let (a_x, a_y) be the un-optimal pair. We can always construct an Euler cycle going through all the edges of the Steiner tree twice so that offices a_x and a_y will be visited adjacently. To do that, we start at any office and go through edges as in dfs (left picture). Then, we change the path from a_x to a_y (right picture).



Since all paths during daytime are at least as good as at nighttime and (a_x, a_y) is un-optimal, we can construct a better tour during daytime, which breaks the requirement of the problem. We will use claim 7.2 to supersede the problem requirement.

To solve this subtask, we go over all the sets of buildings and check if all the pairs are optimal for each.

Time complexity: $O(2^{n^2} * n^4)$.

Subtask 8: $T = 1$; $N, M \leq 50$

Now we need to figure out what kind of sets are optimal.

Claim 8.1: we cannot have two leaves in the Steiner tree with an edge in the same direction.

Proof 8.1: assume the opposite. WLOG, let A and B be the leaves of the Steiner tree (they must be office buildings because otherwise, the Steiner tree would not include them), which both have an edge to the left. The tour in the nighttime then will look like this: $\dots LR \dots LR \dots$ (here LR corresponds to entering and exiting the offices A and B). Now if we look at the path $(A \rightarrow B) \rightarrow R \dots L$, it becomes clear that (A, B) is an un-optimal pair.

Claim 8.2: The Steiner tree can have at most four leaves.

Proof 8.2: We use the pigeonhole principle with the previous claim.

Claim 8.3: The Steiner tree can have at most two vertices with degrees higher than two.

Proof 8.3: assume the opposite. Since there are at most four leaves, the sum of the degrees of the Steiner tree would be at least $3 + 3 + 3 + 2 + \dots + 2 + 1 + 1 + 1 + 1 = 2n - 1$ which is impossible.

Let us name two vertices with > 2 degrees A and B .

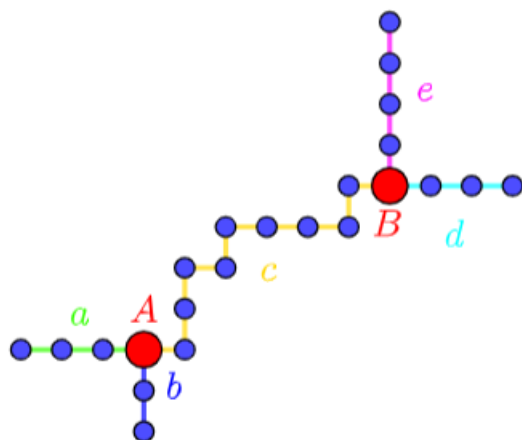
If there is only one such vertex — let's call it central — its degree is either 3 or 4. If it's 4, then let it be both A and B . If it's 3, then we have three leaves in the Steiner tree which make an optimal triplet. Exactly one of those leaves has more than one direction of edges in the path towards the center. Let's name that building B and the center A .

If there are no such vertices, then the Steiner tree has precisely two leaves; then they will be A and B . Let us call A and B focuses.

Observation 8.1: The focuses can't have exactly two neighbors in the Steiner tree.

So now we have fixed the focuses, and as in subtask 6, we will count the number of optimal sets for each fixed pair separately and then sum everything up to get the answer.

Let's define the maximal Steiner tree for a pair (A, B) as an optimal set with the largest size which also contains them as the focuses.



This is how the maximal Steiner tree looks like. We have two main vertices, and five paths, four of which are straight lines. Some of these paths could have length zero. We can choose the vertices from those five paths to form an optimal set. Because of observation 8.1, It is necessary to choose vertices either on both or none of the paths a and b . Same for d and e . Also we can exclude A from the set only then we choose vertices from both a and b . Same for B . Other than that there are no constraints on an optimal subset with A and B as focuses. We can easily count these variants using just *dpLine*. Also it

is important to not count the variants with A and B in the same row/column twice.

Time complexity: $O((NM)^2)$.

Subtask 9: $T = 1$; Road descriptions do not contain character '3'.

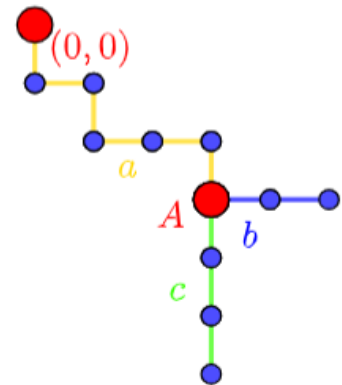
Claim 9.1: Road descriptions do not contain character '0' (apart from up-left corner).

Proof: challenge for the reader 😊.

Since all buildings except $(0, 0)$ have an edge up or to the left, any pair $((0, 0), (x, y))$ is optimal, so all maximal Steiner trees contain $(0, 0)$ as a leaf. It is easy to see (but wordy to prove) that all maximal Steiner trees look similar to the one in the picture.

The solution for the previous subtask is enough to solve this one because one of the focuses is always in $(0, 0)$.

Time complexity: $O(NM)$



Subtask 10: $T = 1$

We do the same as in subtask 8 but using dp. We do it for both orientations of the Steiner tree (from up-left to down-right and from up-right to down-left). Since both dp-s are identical, we will consider only the first one. $dp[i][j]$ is the number of optimal sets that contain (i, j) and vertices which have coordinates not more than (i, j) . Using dp and $dpLine$ we can count the answer.

Time complexity: $O(NM)$.

Square Grid Puzzle

Task Author: Host Scientific Committee

A straightforward approach is to solve the puzzle row by row. To achieve this, we examine the numbers in the target row and attempt to place them in the same row in some order. To do so we can use Right moves. However, if there are two or more such numbers in the same column, this won't immediately be possible. To address this, we can first execute Down moves, redistributing the necessary numbers across different columns. Finally, we use additional Down moves to arrange each row in the desired order.

Initial	After Down moves	After Right Moves	After Down moves																																																																																																				
<table><tr><td>3</td><td></td><td></td><td></td><td></td></tr><tr><td>2</td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>4</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td>0</td><td></td></tr><tr><td></td><td>1</td><td></td><td></td><td></td></tr></table>	3					2						4							0			1				<table><tr><td>3</td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>2</td><td></td><td></td></tr><tr><td></td><td>4</td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td>0</td><td></td></tr><tr><td></td><td></td><td></td><td></td><td>1</td></tr></table>	3							2				4							0						1	<table><tr><td>3</td><td>4</td><td>2</td><td>0</td><td>1</td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	3	4	2	0	1																					<table><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	0	1	2	3	4																				
3																																																																																																							
2																																																																																																							
	4																																																																																																						
			0																																																																																																				
	1																																																																																																						
3																																																																																																							
		2																																																																																																					
	4																																																																																																						
			0																																																																																																				
				1																																																																																																			
3	4	2	0	1																																																																																																			
0	1	2	3	4																																																																																																			

This approach requires less than $2N^2$ moves and is enough to get 50% of the points.

To achieve a perfect score, it's natural to solve each row simultaneously. The last two steps remain unchanged and each can be completed in exactly N moves, leaving fewer than N moves for the initial step. Start by finding the correct target configuration, where rows are rearranged so that numbers from the same target row are never in the same column. Execute Down moves to reach this configuration. Notably, the last row doesn't require rearranging, so only $N - 1$ moves are necessary for this step, totaling $3N - 1$ moves.

Due to the small grid size, finding the correct configuration can be accomplished through a full search. It is beneficial to perform this search column by column, rather than row by row. Doing so allows us to prove that, regardless of how the initial columns are filled, a valid solution for the subsequent ones will always exist. Each column necessitates testing at most $N!$ possibilities. In practice, the first valid solution is often found significantly faster.

Infection

Task Author: Hugo Peyraud-Magnin (France)

Subtask 1. Here we have $n \leq 300$ so we can fix each possible source separately and then find all reachable vertices from a single vertex separately. Finding reachable vertices from a single vertex for a fixed source can be done with a single DFS. So total complexity will be $O(n^3)$.

Subtask 2. $R = 0$ means that only infected vertex is the source. If $M > 0$ then any vertex will be reachable from any other vertex excluding the source and answer will be $n * (n - 1)/2$ for every source. If $M = 0$ we can not pass through the source but any other pair will be reachable. We need to count the sizes of subtrees of every vertex which can be done with a simple DFS. Then we know that for one child of the source with subtree size sz amount of pairs will be $sz * (sz - 1)/2$ and we can easily sum up these values for every child. Amount can be similarly counted for the rest of the tree size of which will be equal to $n - sz_{source}$. Complexity will be $O(n)$.

Subtask 3. For $M = 2R + 1$ any non-infected vertex can be reached from any other non-infected vertex. So we just have to count the number of vertices that will not be infected for every source. We already know the amount of vertices outside the subtree of the source from previous subtask. For non-infected vertices in the subtree of the source we can sum up subtree sizes of the vertices that are $R+1$ deeper from the source. Which we can do in DFS if we maintain a list of ancestors and add current size to $R+1$ -th ancestors count of non-infected vertices. This still needs a single DFS and complexity is $O(n)$.

Subtask 4. $M = 2R - 1$ means that we can not go through the source when we are trying to go from a vertex inside the subtree of the source to a vertex inside the subtree of the source. In this case we can solve the problem independently for each child of the source. Solving this for a single child would be equivalent to solving the previous subtask for $R - 1$. So we can just use the previous solution for $R - 1$ and sum the answers of the children which can be done simultaneously. Complexity $O(n)$.

Subtask 6. To complete the solution we can build on what we did in subtask 4. In subtask 4 highest points we could have passed through when going from inside the subtree of the source to inside the subtree of the source were children of the source. That is not the case anymore but we can measure exactly what will be the highest point. If we can pass through at most M infected vertices then we can go up by at most $M/2$ from the bottom because we can't stop on an infected vertex. So we are interested in vertices with depth $R - M/2$ from the source. Since these are the highest points we can pass through we can count them independently. And for each one of them we can solve subtask 3 for radius $M/2$. And then we need a sum of these values which we can do similarly to subtask 4. This gives us all the possible pairs inside the subtree of the source. We should add all pairs from outside the subtree and for mixed (one inside, one outside) pairs if $M > R$ all pairs are reachable otherwise none are reachable. This all can still be done with a single DFS so final complexity is $O(N)$

Team Building

Task Author: Host Scientific Committee

Subtask 1

We can try all $N!$ permutations, simulate the whole process and pick the best result.

Subtask 2

Notice that the skills of the first and the last programmers don't affect anything, so the two least skilled ones should be the first and the last.

The natural extension would be to put the next two least skilled ones next to them, etc.

Another possibility is to use the solution for Subtask 1 on different examples and notice the same pattern about the optimal order.

The most straightforward way to implement this is to sort all the skills for each day and simulate the process to compute the strength.

Subtask 3

We should either make the simulation faster or find another way which is a step forward to a better solution.

Notice that the programmer that is hired K -th will affect the motivations of the previous $K - 1$ programmers.

Those motivations will increase their workrates for each following programmer, $N - 1 - K$ in total.

That implies that the skill level in position K has multiplier $K \cdot (N - K - 1)$ which means total strength can be computed in linear time after sorting.

Subtask 4

To solve the subtasks for larger N and Q , we can no longer afford to recompute the answer for each query.

Instead, we need to maintain the sorted list of all current skill values.

This subtask can be solved as part of either Subtask 5 or Subtask 7.

Subtask 5

Since there are few possible skill values, instead of maintaining the whole list, it's enough to maintain the frequencies of different values.

Instead of linear strength computation, it's possible to go through the frequencies and compute the sum of products using some kind of prefix sums precomputed for multipliers.

Subtask 6

Since only one element gets updated, the rest of the sorted list doesn't change.

We can either precompute the sums of products for each prefix and suffix to handle adding the first element in the middle in constant time (with binary search needed to find that position), or we can try all the values of the first element in increasing order and compute sum changes between successive insertions in amortized linear time.

Subtask 7

We have a single element of the sorted list of integers change by 1. Since the equal elements are indistinguishable, we can choose to modify either the leftmost or rightmost element of the equal values that's being changed, which allows us to keep the elements in the list sorted without any movements. This allows logarithmic queries if we maintain the positions for each possible value in the sorted data structure (like `std::set`).

Subtask 8

We will need a more complex data structure to solve the general case efficiently, which will be a segment tree where each element computes the necessary values for each segment to be able to merge the segments, and the answer will be in the root of the tree after each update.

We'll prepare the segment tree for $N + Q$ elements, one for each possible skill value (initial or updated).

At each step, we will have N active elements: each update will remove one and add another.

The sorted list of multipliers $K \cdot (N - K - 1)$ has the values $K = 0$ and $K = N - 1$ first and they have the same value, the same thing applies to $K = 1$ and $K = N - 2$, etc.

There are two copies of each multiplier (except maybe the last value).

To simplify things we can build two segment trees - for odd positions and even positions separately.

The merging step should check the active element count in the left segment and, depending on the parity, merge with either the odd part or even right part. In each of the trees, the multipliers are $C[K] = K \cdot (N - K - 1)$ for $K \geq 0$.

Suppose the sorted skill values are $X[K]$ for $0 \leq K$. The value we need to compute is the sum of $X[K] \cdot C[K] = X[K] \cdot (K \cdot (N - K - 1)) = (N - 1) \cdot X[K] \cdot K - X[K] \cdot K^2$

Since $N - 1$ is constant, it's enough to maintain the sums of $X[K] \cdot K$ ($SUM1$) and $X[K] \cdot K^2$ ($SUM2$).

To achieve this, we also maintain the sums of $X[K]$ (SUM) and the element count for each segment. With this information, these sums can be updated using straightforward calculations:

$$\begin{aligned} SUM(L, R) &= X[L] + X[L + 1] + \dots + X[L + M] + \dots + X[R] = \\ &= SUM(L, M) + SUM(L + M, R) \end{aligned}$$

$$\begin{aligned} SUM1(L, R) &= X[L] \cdot 0 + X[L + 1] \cdot 1 + \dots + X[L + M] \cdot M + \dots + X[R] \cdot (R - L) = \\ &= SUM1(L, M) + SUM1(L + M, R) + M \cdot SUM(L + M, R) \end{aligned}$$

$$\begin{aligned} SUM2(L, R) &= X[L] \cdot 0^2 + X[L + 1] \cdot 1 + \dots + X[L + M] \cdot M^2 + \dots + X[R] \cdot (R - L)^2 = \\ &= SUM2(L, M) + SUM2(L + M, R) + M^2 \cdot SUM(L + M, R) + 2 \cdot M \cdot SUM1(L + M, R) \end{aligned}$$

This approach allows for efficient solution with a time complexity of $O((N + Q) \log(N + Q))$.